

Game — Loop Tracer (`breakpoint()` in a `for` loop)

A short exercise in using Python's interactive debugger (`pdb`) to step through a `for` loop one iteration at a time, watching variables come into existence as the loop runs.

The Script

```
python

# loop_tracer.py
scores = [70, 85, 60, 95]
bonus = 5
total = 0

for i, s in enumerate(scores):
    breakpoint() # ← pauses
    boosted = s + bonus
    total += boosted

print("Total:", total)
```

Your Mission

1. Run the script. It pauses every time the loop hits `breakpoint()`.
 2. At each pause, type `(p i)` and `(p s)` to see the current index and score.
 3. Type `(p boosted)` — notice it doesn't exist **yet** (`NameError`) until you run `(n)` twice.
 4. Type `(c)` to continue to the **next** iteration — repeat for all 4 items.
 5. Fill in the tracking table below as you go!
-

Trace Table — fill in as you step through!

Iter.	<code>(i)</code>	<code>(s)</code>	<code>(boosted)</code> (<code>s+5</code>)	<code>(total)</code> (running)
-------	------------------	------------------	---	--------------------------------

#1

#2

Iter.	i	s	boosted (s+5)	total (running)
#3				
#4				

Files

File	Role
loop_tracer.py	The script containing the loop and breakpoint().

Requirements

- Python 3.7+ (for built-in breakpoint() support)

Running

```
bash

python loop_tracer.py
```

Execution will pause inside the loop on every iteration, dropping you into an interactive (Pdb) session each time.

Useful pdb commands for this exercise

Command	Effect
p <expr>	Print the value of <expr> (e.g., p i, p s, p boosted)
n	Next line (step over) — use this to watch boosted get created
c	Continue execution until the next breakpoint() (i.e., the next iteration)
l	List source code around the current line
q	Quit the debugger

Why `p boosted` Fails at First

When `pdb` pauses, it stops **before** the line it's sitting on has run. Since `breakpoint()` is the first line inside the loop body, `boosted` hasn't been assigned yet on that pause — Python simply doesn't know about it. Step forward with `n` past the `boosted = s + bonus` line, and `p boosted` will work. This is a good way to build intuition for how Python creates variables as execution reaches each assignment, not all at once.